

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1418

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 15

Total Marks:30

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I _____ (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines
specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

SHORT ANSWER TEST

Q1. What is an ambiguous grammar? State the formal condition for ambiguity and its impact on compiler design.

Q2. What is a sentential form? Distinguish between left-sentential and right-sentential forms with examples.

Q3. Define a production rule. What is an epsilon (ϵ) production and when is it used in a grammar?

Q4. Check whether $E \rightarrow E + E \mid E * E \mid \text{id}$ is ambiguous. Provide two distinct parse trees for: $\text{id} + \text{id} * \text{id}$.

Q5. $S \rightarrow aSb \mid \epsilon$. Derive the string 'aabb' using rightmost derivation from: $S \rightarrow aSb \mid \epsilon$. Compare with leftmost derivation.

Q6. $E \rightarrow E + T \mid T, T \rightarrow T * F \mid F, F \rightarrow (E) \mid \text{id}$. Remove left recursion from: $E \rightarrow E + T \mid T, T \rightarrow T * F \mid F, F \rightarrow (E) \mid \text{id}$.

Q7. $A \rightarrow aAb \mid aAc \mid d$. Apply left factoring to: $A \rightarrow aAb \mid aAc \mid d$. Show all steps.

Q8. What is recursive descent parsing? State its advantages, limitations, and when it is applicable.

Q9. $S \rightarrow aS \mid bA, A \rightarrow c \mid \epsilon$. Write recursive descent parsing procedures for: $S \rightarrow aS \mid bA, A \rightarrow c \mid \epsilon$.

Q10. $S \rightarrow aABb, A \rightarrow c \mid \epsilon, B \rightarrow d \mid \epsilon$. Compute FIRST and FOLLOW for: $S \rightarrow aABb, A \rightarrow c \mid \epsilon, B \rightarrow d \mid \epsilon$. Show all steps. **Q11.** $S \rightarrow aAB, A \rightarrow b \mid \epsilon, B \rightarrow cd$ – Trace predictive parsing for: $S \rightarrow aAB, A \rightarrow b \mid \epsilon, B \rightarrow cd$ on input 'abcd'. Show stack, input, and action at each step.

Q12. What is a handle in bottom-up parsing? Explain how a handle is identified during shift-reduce parsing.

Q13. Construct operator precedence relations for: $E \rightarrow E + E \mid E * E \mid \text{id}$. Build the full precedence table.

Q14. What is an LR parser? List its four components and explain the general LR parsing algorithm.

Q15. Write a YACC specification with semantic actions to evaluate arithmetic expressions with +, *, and integers.

Bottom of Form **RUBRICS FOR**

CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Name: N. Sumanath Reddy

Register no.: 192525083

Course code: CSA1418

Course Name: Compiler Design.

① Ambiguous grammar: The ambiguous grammar is a grammar in which at least one string has more than one parse tree.

Formal condition: A grammar G is ambiguous if there exists a string $w \in L(G)$ that has two or more distinct parse trees.

• Impact on compiler design: multiple interpretations of a program causes parsing difficult.

• Leads to incorrect code generation.

• Hence, compiler grammars should be unambiguous.

② Sentential form: A string of terminals & non-terminals derived from the symbol

Left sentential form:

• Obtained using leftmost derivation.

• Always non-terminal first.

Right sentential form using right most derivation.

- obtaining or obtain
- Always expands the right most non-terminal first.

Example: Grammar

- $S \rightarrow AB$
- $A \rightarrow a$
- $B \rightarrow b$

Leftmost : $S \Rightarrow AB \Rightarrow \cancel{A}B \Rightarrow ab$

Rightmost : $S \Rightarrow AB \Rightarrow Ab \Rightarrow ab$

A production rule is a grammar rule of the form $A \rightarrow a$ where A is a non-terminal and a is a string of terminals and/or non-terminals.

- An ϵ production is a production that derives the empty string : $A \rightarrow \epsilon$

Uses:

- Represents optional constructs.
- Allows empty strings a language.
- simplifies recursive grammar definition.

① yes, the grammar is ambiguous because the string $id + id * id$ has two parse trees.

1. $(id + id) * id$.

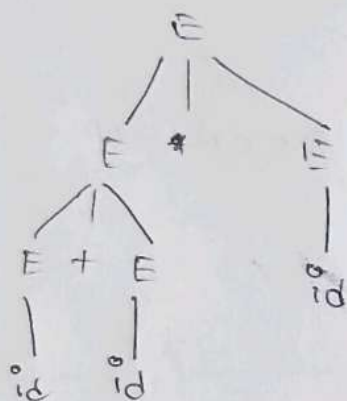
2. $id + (id * id)$.

1) $(id + id) * id$

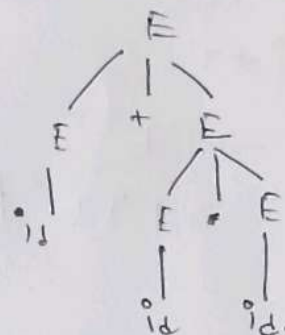
$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow id$$



2) $id + (id * id)$.



$$S \rightarrow asb \mid \epsilon$$

Explanation

$$S \Rightarrow asb$$

$$\Rightarrow aasbb$$

$$\Rightarrow aa\epsilon bb$$

$$\Rightarrow aa\ bb$$

$$S \Rightarrow asb$$

$$asb \Rightarrow aasbb$$

$$aasbb \Rightarrow aa\epsilon bb$$

$$aa\epsilon bb \Rightarrow aa\ bb$$

$$S \Rightarrow asb \Rightarrow aasbb \Rightarrow aa\epsilon bb \Rightarrow aa\ bb.$$

⑥

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid id \end{aligned}$$

STEP 1:

$$E \rightarrow E + T \mid T$$

$$A \rightarrow A\alpha \mid \beta$$

$$\alpha = +T$$

$$\beta = T$$

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

STEP 2:

$$T \rightarrow T * F \mid F$$

$$\alpha = *F$$

$$\beta = F$$

$$T = FT'$$

$$T' = *FT' \mid \epsilon$$

STEP 3: F has no left recursion!

$$F \rightarrow (E) \mid id$$

Final:

$$E \Rightarrow TE'$$

$$E' \Rightarrow +TE' \mid \epsilon$$

$$T \Rightarrow FT'$$

$$T' \Rightarrow *FT' \mid \epsilon$$

$$F \Rightarrow (E) \mid id$$

⑦

Given that:

$$A \rightarrow aAb | aAC | d$$

$$A \rightarrow aA' | d \text{ (Factor out a)}$$

$$aAb \text{ (After remain a)}$$

$$Ab \text{ (Remaining part)}$$

$$aAC \text{ (from)}$$

$$aAC \text{ (Remaining part)}$$

$$A' \rightarrow ab | aAC \text{ (So)}$$

$$A \rightarrow aA' | d$$

$$A' \rightarrow Ab | aAC$$

Recursive Descent parsing: A top-down parsing technique in which each non-terminal is implemented as a recursive function.

Advantages:

- * simple to implement.
- Easy to understand and debug.
- May Efficient for LL(1) grammars.

Limitation:-

- * cannot handle left recursive grammar
- * may require backtracking
- * not suitable for ambiguous grammar

Used when:

- * The grammar is LL(1)
- * Building simple compilers and interpreters

void S ()

```
{  
    match ('a');  
    A ();  
    B ();  
}
```

void A ()

```
{  
    if (input == 'a')  
    {  
        match ('a');  
        A ();  
    }  
}
```

void U

```
{  
    if (input == 'b')
```

$\{$
 $\text{match ('b')};$
 $\text{BCD};$
 $\}$
 $\}$

(10) $S \rightarrow aABb$
 $A \rightarrow c | \epsilon$
 $B \rightarrow d | \epsilon$

Step 1: Find first sets.

$\text{First}(S) = \{a\}$

$\text{First}(A)$

$A \rightarrow c | \epsilon$

$\text{FIRST}(A) = \{c, \epsilon\}$

$\text{First}(B)$

$B \rightarrow d | \epsilon$

$\text{FIRST}(B) = \{d, \epsilon\}$

Non terminal

S

A

B

first

$\{a\}$

$\{c, \epsilon\}$

$\{d, \epsilon\}$

Step 2: Find follow sets

$\text{Follow}(S) = \{\$ \}$

$\text{Follow}(A)$

$S \rightarrow aABb$

After A comes
Bb

$\text{FIRST}(B) = \{d$

$\text{Follow}(A) = d$

$S \rightarrow aABb$

$\text{Follow}(B) = \{$

follow

$\{\$ \}$

$\{d, b\}$

$\{b\}$

⑪ Predictive Parsing

$S \rightarrow aAB$

$A \rightarrow b|e$

$B \rightarrow cd$

Input = $abcd\$$

Stack	input	Action.
$S\$$	$abcd\$$	$S \rightarrow aAB$
$aAB\$$	$abcd\$$	match a
$AB\$$	$bcd\$$	$A \rightarrow b$
$bB\$$	$bcd\$$	match b
$B\$$	$cd\$$	$B \rightarrow cd$
$cd\$$	$cd\$$	match c
$d\$$	$d\$$	match d
$\$$	$\$$	Accept.

② A handle is a substring the matches the right-hand side of a production & can be reduced to its left-hand side.

Shift-Reduce Parsing Steps:-

1. Shift input symbols onto the stack
2. Identify the handle.
3. Reduce it to the corresponding non-terminal
4. Accept the input.

13

Operator Precedence Relations

$E \rightarrow E + E$
 $E \rightarrow E * E$
 $E \rightarrow id$

$* > +$
 $+ < *$
 $id > +$
 $id > *$
 $\$ < id$
 $id > \$$

Precedence table:

	<u>id</u>	<u>+</u>	<u>*</u>	<u>\$</u>
<u>id</u>	-	>	>	>
<u>+</u>	<	>	<	>
<u>*</u>	<	>	>	>
<u>\$</u>	<	<	<	=

4

An LR parser is a bottom-up parser that reads input left to right & produces the rightmost derivation in reverse.

Four components:

1. Input Buffer.
2. stack
3. Parsing Table
4. parser driver.

Algorithm (short):

- Initialize stack.
- Read input symbol.

15

% {

include <stdio.h>

include <stdlib.h>

% }

% token Num

%%

expr : expr '+' term { ## = #1 + #3 ; }

{ ## = #1 ; }

|
:
;

term : term '*' factor { ## = #1 * #3 ; }

factor

{ ## = #1 ; }

|
:
;

factor : Num

{ ## = #1 ; }

%%

int

main ()

{ printf ("Enter expression: \n");

if parse ();

return 0;

}

int yyerror (char *)

{ printf ("Syntax Error \n");

return 0;

}